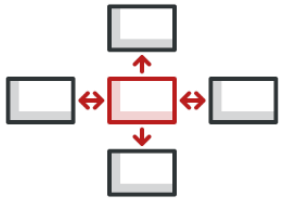
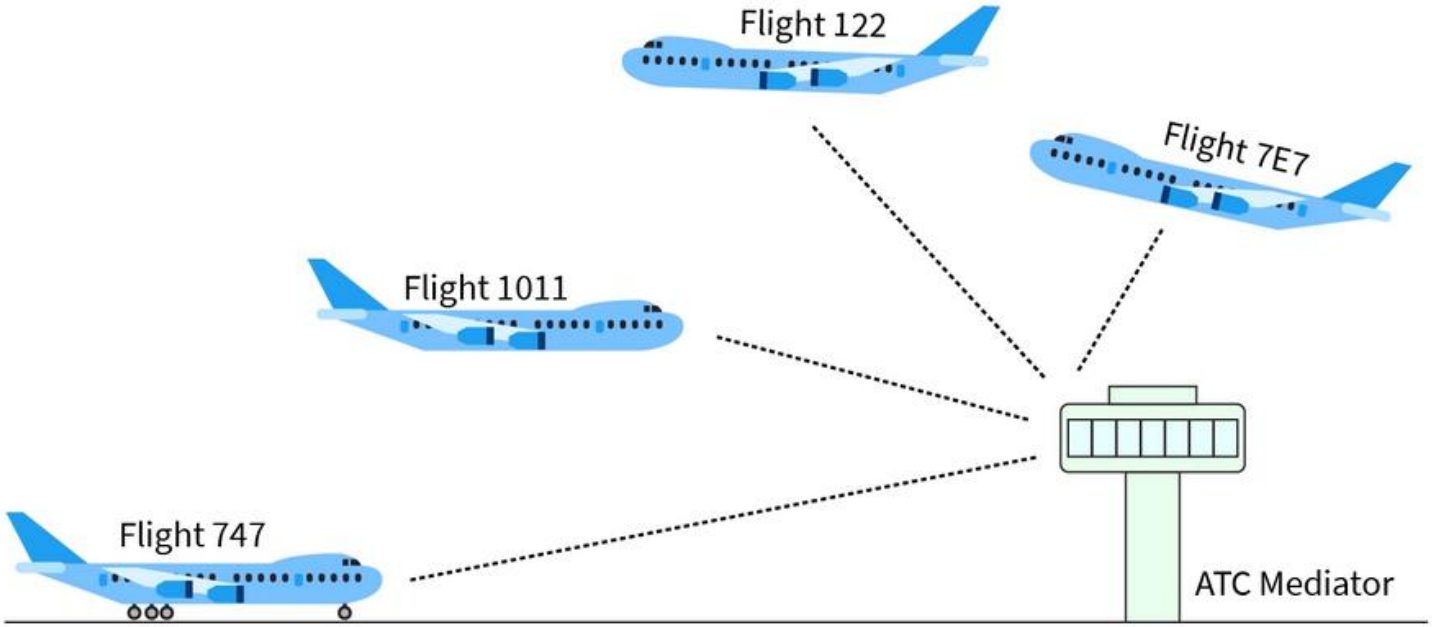
Поведенческие паттерны. Посредник



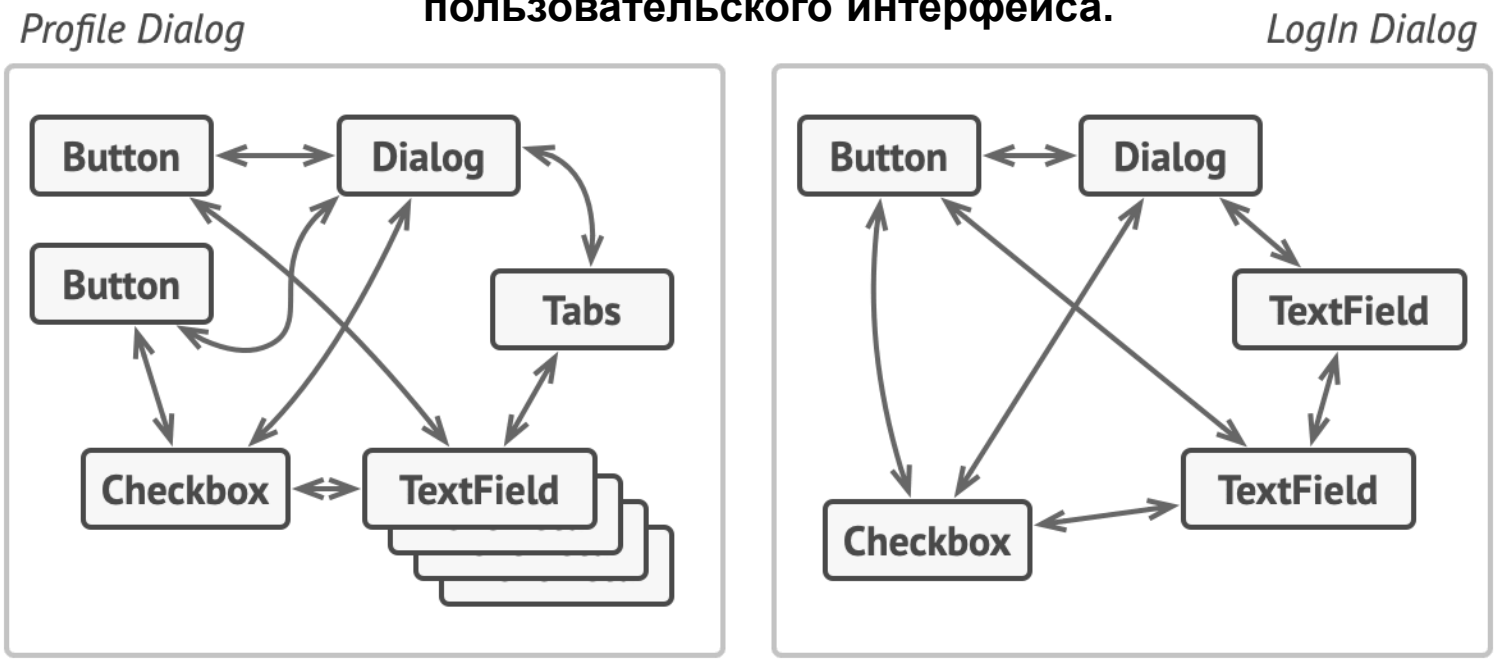
Паттерн Посредник
Mediator (Intermediary)



Mediator – поведенческий паттерн, реализует *объект-посредник,* скрывающий способ взаимодействия множества других объектов-коллег.

Посредник обеспечивает слабую связанность системы, избавляя объектов-коллег от необходимости явно ссылаться друг на друга, позволяя тем самым независимо изменять взаимодействия между объектами-коллегами.

Неструктурированные связи между элементами пользовательского интерфейса.



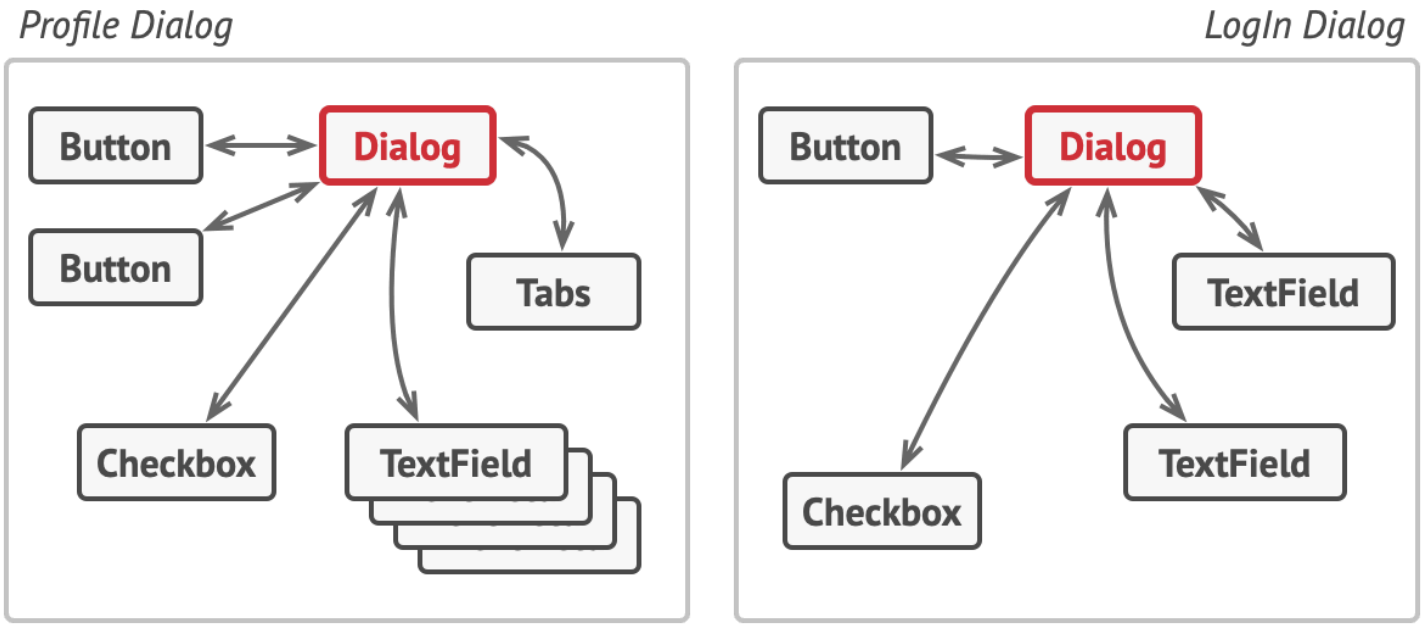
Сильная связь между элементами диалога, приводит к необходимости редактирования кода элементов при изменении диалогов



Решение

Паттерн **Посредник** заставляет объекты общаться не напрямую друг с другом, а через отдельный объект-посредник, который реализует перенаправление запросов.

Благодаря этому, компоненты системы будут зависеть только от посредника, а не от множества других компонентов.



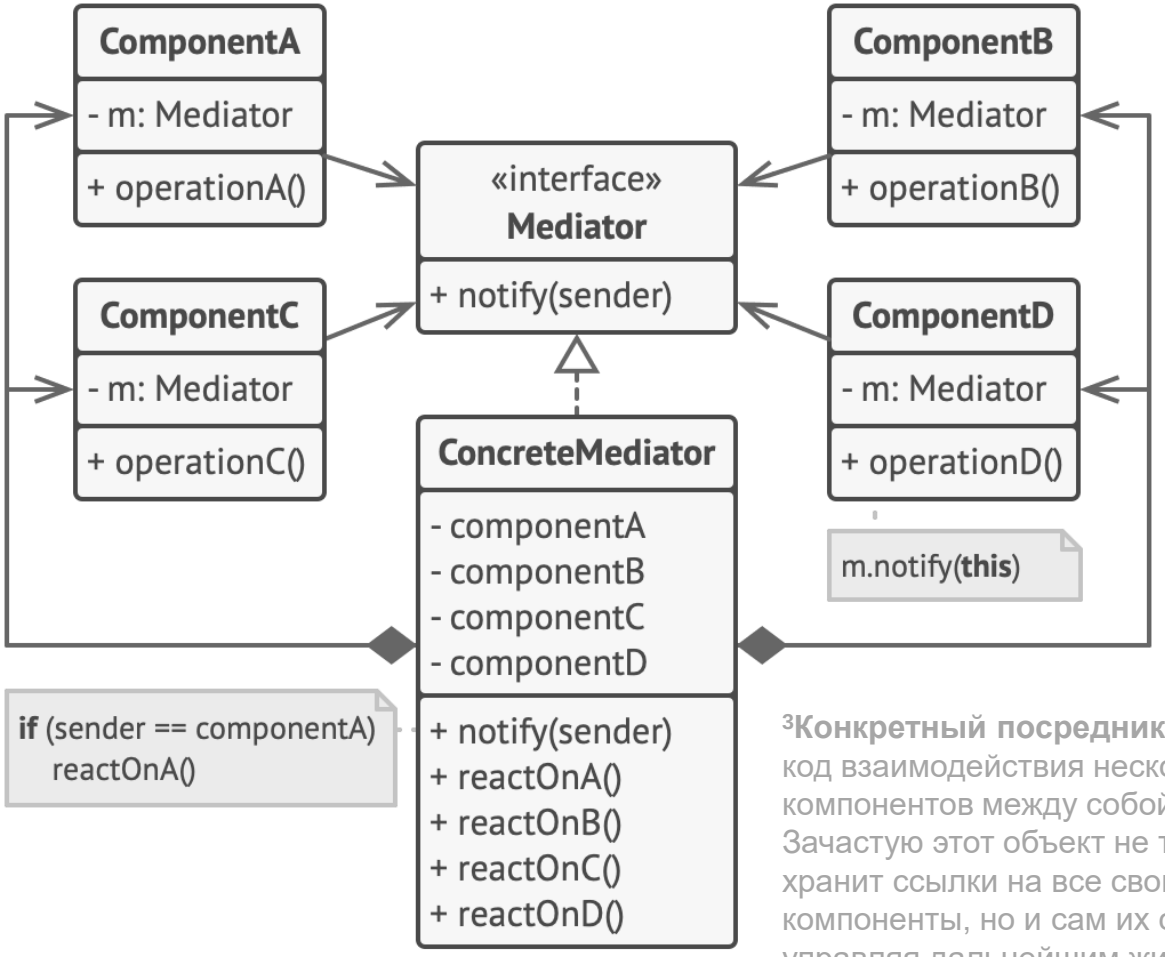
Посредник скрывает в себе все сложные связи и зависимости между классами отдельных компонентов программы.

Чем меньше связей имеют классы, тем проще их изменять, расширять и повторно использовать.

Структура паттерна

[1]**Компоненты** — это разнородные объекты, содержащие бизнес-логику программы. Каждый компонент хранит ссылку на объект посредника, но работает с ним только через абстрактный интерфейс посредников. Благодаря этому, компоненты можно повторно использовать в другой программе, связав их с посредником другого типа.

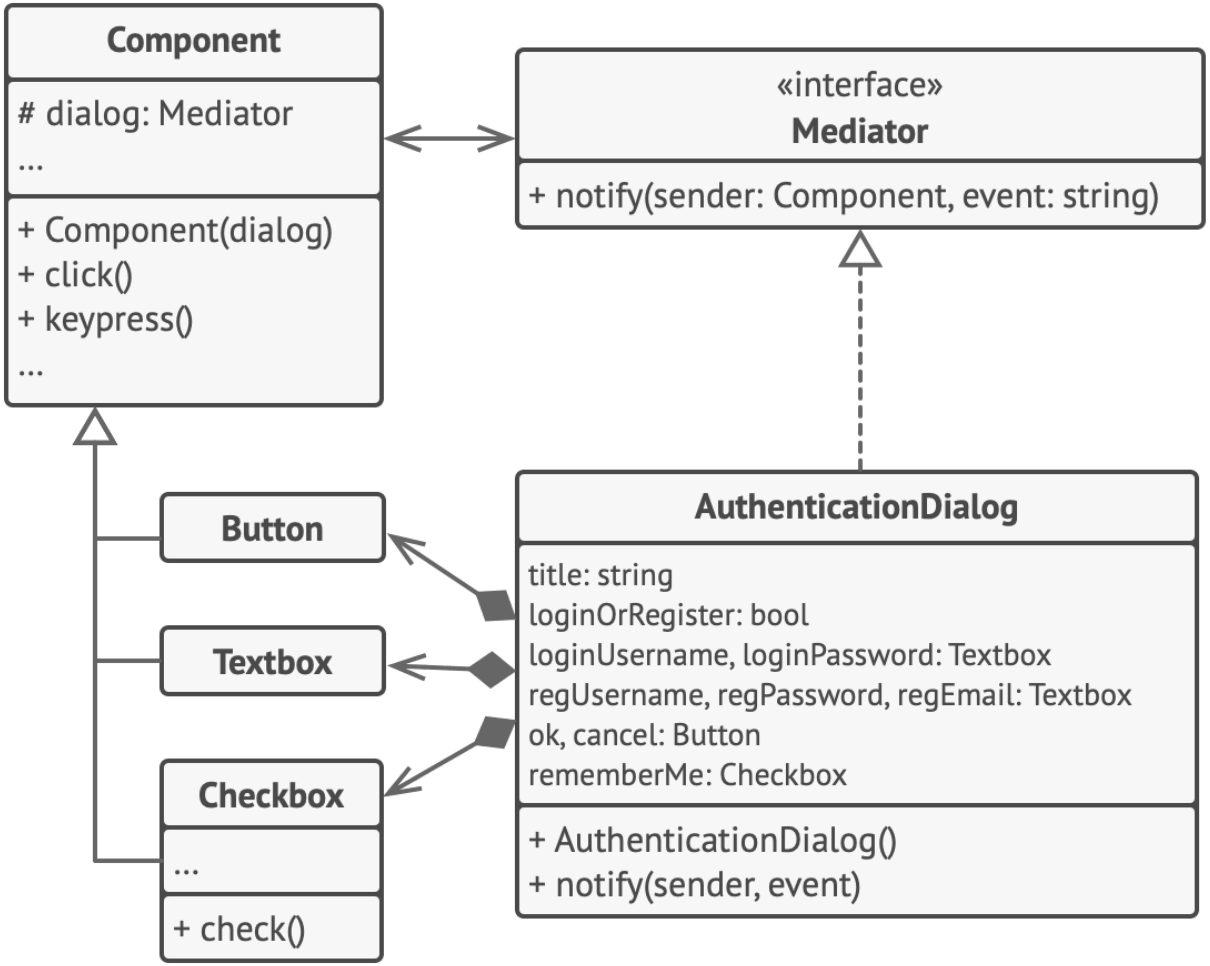
[2]**Посредник** определяет интерфейс для обмена информацией с компонентами. Обычно хватает одного метода, чтобы оповещать посредника о событиях, произошедших в компонентах. В параметрах этого метода можно передавать детали события: ссылку на компонент, в котором оно произошло, и любые другие данные.



[3]**Конкретный посредник** содержит код взаимодействия нескольких компонентов между собой. Зачастую этот объект не только хранит ссылки на все свои компоненты, но и сам их создаёт, управляя дальнейшим жизненным циклом.

[4]Компоненты не должны общаться друг с другом напрямую. Если в компоненте происходит важное событие, он должен оповестить своего посредника, а тот сам решит — касается ли событие других компонентов, и стоит ли их оповещать. При этом компонент-отправитель не знает кто обработает его запрос, а компонент-получатель не знает кто его прислал.

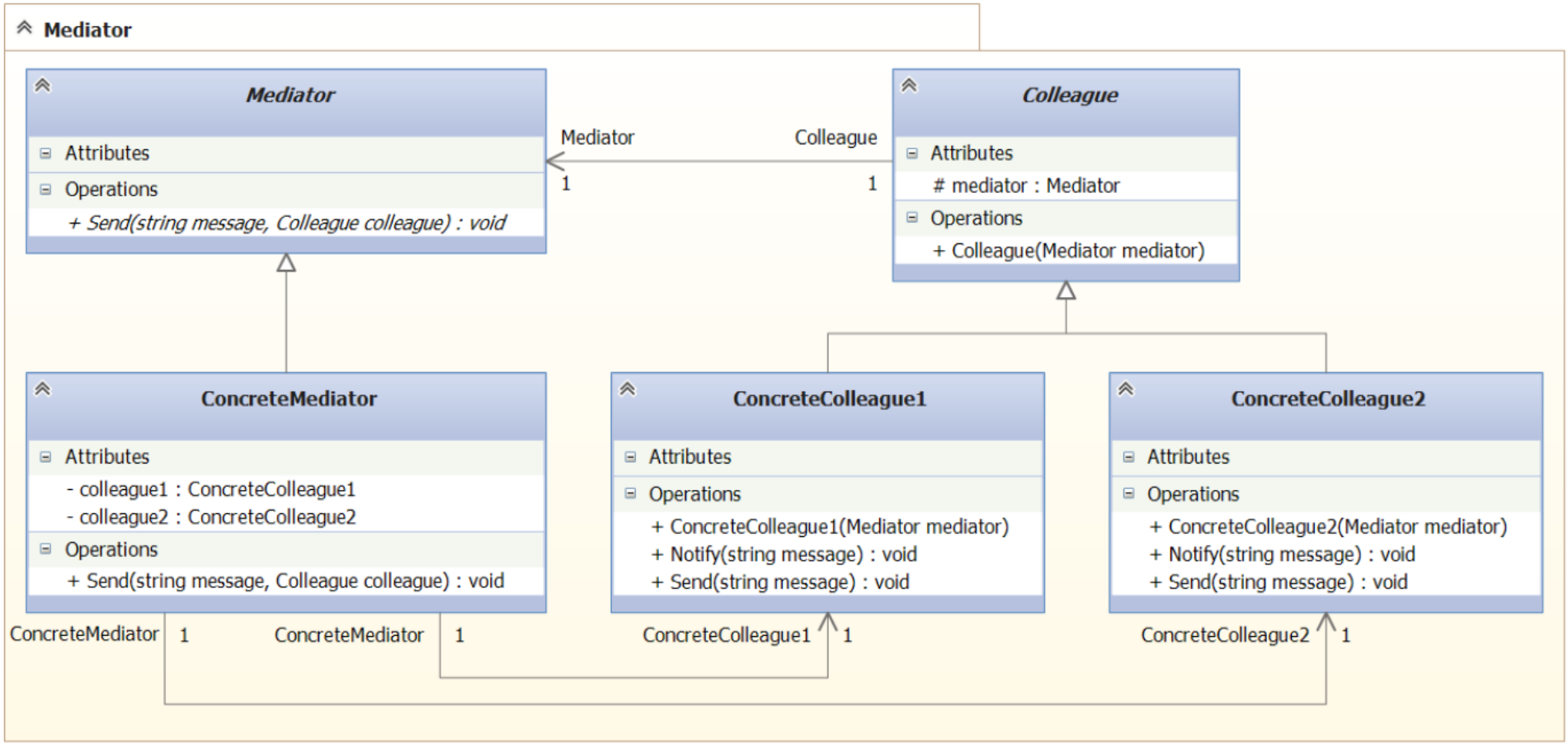
Пример структурирования классов UI-диалогов.



Реагируя на действия пользователей элементы не взаимодействуют напрямую, а лишь *уведомляют посредника* о том, что они изменились.

При получении уведомлений посредник может *перенаправить вызов* тому или иному элементу.

Структура паттерна UML



Структура паттерна на языке C#

```
class Program
{
    static void Main()
    {
        var mediator = new ConcreteMediator();
        var colleague1 = new ConcreteColleague1(mediator);
        var colleague2 = new ConcreteColleague2(mediator);

        mediator.Colleague1 = colleague1;
        mediator.Colleague2 = colleague2;
        colleague1.Send("Message 1");
        colleague2.Send("Message 2");
    }
}
```

`Mediator` - **Посредник**:
Предоставляет интерфейс для организации процесса по обмену информацией между объектами типа `Colleague`.

```
abstract class Mediator
{
    public abstract void Send(string msg, Colleague colleague);
}
```

```
class ConcreteMediator : Mediator
{
    public ConcreteColleague1 Colleague1 { get; set; }
    public ConcreteColleague2 Colleague2 { get; set; }

    public override void Send(string msg, Colleague colleague)
    {
        if (Colleague1 == colleague)
            Colleague2.Notify(msg);
        else
            Colleague1.Notify(msg);
    }
}
```

`ConcreteMediator` - **Конкретный посредник**:
Реализует алгоритм взаимодействия между объектами-коллегами

```
abstract class Colleague
{
    protected Mediator mediator;

    public Colleague(Mediator mediator)
    {
        this.mediator = mediator;
    }
}
```

`Colleague` - **Коллега**:
Предоставляет интерфейс для организации процесса взаимодействия объектов-коллег с объектом типа `Mediator`.

```
class ConcreteColleague1 : Colleague
{
    public ConcreteColleague1(Mediator mediator)
        : base(mediator)
    {
    }

    public void Send(string message)
    {
        mediator.Send(message, this);
    }

    public void Notify(string message)
    {
        Console.WriteLine(message);
    }
}
```

```
class ConcreteColleague2 : Colleague
{
    public ConcreteColleague2(Mediator mediator)
        : base(mediator)
    {
    }

    public void Send(string message)
    {
        mediator.Send(message, this);
    }

    public void Notify(string message)
    {
        Console.WriteLine(message);
    }
}
```

`ConcreteColleague` - **Конкретный коллега**:
Каждый объект-коллега знает только об объекте-медиаторе. Все объекты-коллеги обмениваются информацией только через посредника (медиатора).

Отношения между участниками

Отношения между классами

- Класс ConcreteMediator связан связью отношения наследования с абстрактным классом Mediator и связями отношения ассоциации с классами ConcreteColleague.
- Абстрактный класс Colleague связан связью отношения ассоциации с абстрактным классом Mediator.
- Конкретные классы ConcreteMediator связаны связью отношения наследования с абстрактным классом Colleague.

Отношения между объектами

- Объекты-коллеги (ConcreteColleague) посылают запросы объекту-посреднику (ConcreteMediator), а он в свою очередь перенаправляет полученные запросы другим объектам-коллегам, тем самым организуя процесс взаимодействия между объектами-коллегами.

Применение

1. Когда сложно менять классы из-за того, что они имеют множество неструктурированных связей с другими классами.

Посредник позволяет поместить все эти связи в один класс, в результате чего рефакторинг кода упрощается.

2. Когда невозможно повторно использовать класс, поскольку он зависит от множества других классов.

После применения паттерна компоненты теряют прежние связи с другими компонентами, а всё их общение происходит косвенно, через объект-посредник.

3. Когда приходится создавать множество подклассов компонентов, чтобы использовать одни и те же компоненты в разных контекстах.

Если раньше изменение отношений в одном компоненте могли повлечь за собой лавину изменений во всех остальных компонентах, то теперь достаточно создать подкласс посредника и поменять в нём связи между компонентами.

Шаги реализации

1. **Найти группу тесно связанных классов**, отвязав которые друг от друга, можно получить некоторую пользу. Например, чтобы повторно использовать их код в другой программе.

2. **Создать общий интерфейс посредников и описать в нём методы** для взаимодействия с компонентами. В простейшем случае достаточно одного метода для получения оповещений от компонентов.

3.Реализовать этот интерфейс в классе конкретного посредника. Поместить в него поля, которые будут содержать ссылки на все объекты компонентов.

4. **Опционально: переместить код создания компонентов в класс посредника**, после чего он может напоминать фабрику или фасад.

5. **Компоненты тоже должны иметь ссылку на объект посредника**. Связь между ними удобнее всего установить, подавая посредника в параметры конструктора компонентов.

6. **Изменить код компонентов так, чтобы они вызывали метод оповещения посредника**, вместо методов других компонентов. С противоположной стороны, посредник должен вызывать методы нужного компонента, когда получает оповещение от компонента.

Преимущества и недостатки

+ Устраняет зависимости между компонентами, позволяя повторно их использовать.

+ Упрощает взаимодействие между компонентами.

+ Централизует управление в одном месте..

- Посредник может сильно раздуться (содержит множество полей/методов/строк кода).

